Chat Link: https://claude.ai/share/be5ebce5-9372-40a0-aabe-6ebfccc1452d

Chat Content:

Task: Debug and Complete the Arm Wrestling Game

https://github.com/Janavi88/PES1UG24AM120_SE_LAB/tree/main

You have access to my GitHub repository containing the Arm Wrestling game. The project primarily uses game_engine and main.py.

Your task is to **analyze the existing code, fix the deliberate bug, implement all four tasks, and verify that the complete game works correctly.** Do not just provide suggestions or code snippets.Dont make changes directly just return the updated code file.

Step 1: Inspect and Understand the Existing Code

Read game_engine, main.py, and any other relevant project files before making changes.

Identify the game loop, keyboard event handling, arm movement calculations, stamina system, AI force calculations, rendering logic, win/loss detection, and rematch/reset logic.

Understand the existing architecture and preserve its design wherever possible.

Identify the root cause of the inverted arm-push bug before fixing it.

Check how game state and variables are initialized, updated, rendered, and reset.

Do not rewrite the entire game from scratch. Make targeted changes that integrate with the existing implementation.

Step 2: Task 1 — Fix the Inverted Arm Push Bug

Currently, alternating Left and Right arrow key presses move the clasped hands toward the computer's side instead of the player's side.

Requirements:

- Correct the movement direction so valid alternating Left and Right key presses move the hands toward the player's winning threshold.
- Preserve the alternating-input requirement.
- Ensure invalid repeated key presses do not accidentally grant extra pushing force.
- Ensure the arm position remains within its intended bounds.
- Preserve the existing rendering and animation behavior.
- Verify that the player moves toward victory rather than accidentally pinning themselves.

The player's winning threshold is -100.0, and the computer's winning threshold is +100.0. Keep this coordinate convention consistent throughout the game.

Step 3: Task 2 — Implement Dynamic AI Surge

Replace the predictable constant AI force with a dynamic surge-and-cooldown cycle.

Requirements:

- Implement an AI state or equivalent mechanism that alternates between normal resistance, a high-power surge, and a cooldown phase.
- During a surge, increase the AI's pushing force above its normal baseline.
- During cooldown, reduce the AI's pushing force below its normal baseline.
- Make the cycle operate automatically using the existing game loop and elapsed time.
- Use configurable constants for surge duration, cooldown duration, and force multipliers.
- Ensure transitions happen reliably without blocking the game loop or freezing input.
- Keep the cycle fair and playable.

Use the existing AI force system where possible instead of introducing an unrelated mechanism.

Step 4: Task 3 — Implement Exhaustion Warning Indicators

Provide clear visual feedback for both AI surges and player exhaustion.

AI surge warning:

- Display a visible warning when the AI enters its high-power surge phase.
- Make the warning disappear when the surge ends.
- Clearly distinguish the surge state from normal and cooldown states.
- If practical, show a cooldown indicator when the AI becomes temporarily weaker.

Player exhaustion:

- When the player's stamina falls below 10, display a clear exhaustion warning.
- Temporarily prevent the player from generating additional pushing force while exhausted.
- Allow stamina to recover naturally over time.
- Restore the player's ability to push once stamina recovers to the appropriate threshold.
- Prevent rapid repeated key presses from bypassing the exhaustion restriction.

Ensure the indicators are integrated into the existing game UI and remain readable during gameplay. Do not replace the existing arena or stamina bar.

Step 5: Task 4 — Implement Counter-Surge Bonus Resistance

Reward the player for timing a push at the end of an AI surge.

Requirements:

- Detect the transition from the AI's surge phase into its cooldown phase.
- If the player performs a valid push during a small configurable timing window immediately after that transition, activate a counter-surge bonus.
- The bonus must provide:
    a. Double player pushing strength for a temporary duration.

b. Accelerated stamina recovery for the same temporary duration.

- Display a clear visual indicator when the bonus activates and while it remains active.
- Ensure the bonus cannot be activated repeatedly by holding a key or generating duplicate key events.
- Expire the bonus cleanly and restore normal pushing strength and stamina recovery.
- Do not activate the bonus if the player is exhausted or the push is invalid.
- Avoid unintended interactions with the normal stamina system, AI surge state, or game-over state.

Use a configurable timing window and bonus duration. Make sure the bonus is triggered by the surge-to-cooldown transition, not merely by any push during cooldown.

Step 6: Preserve Core Game Functionality

Verify all of the following:

The player must alternate Left and Right arrow key presses to push.

Valid presses move the arm toward -100.0.

The AI continuously pushes toward +100.0.

Rapid pushing drains player stamina.

Stamina recovers naturally over time.

Player input is temporarily disabled below 10 stamina.

The AI surge and cooldown cycle continues correctly during gameplay.

AI surge and exhaustion warnings appear and disappear at the correct times.

A correctly timed counter-surge push activates double pushing strength and accelerated stamina recovery.

Reaching -100.0 produces a player victory.

Reaching +100.0 produces a computer victory.

Pressing R on the Game Over screen starts a fresh rematch.

A rematch resets arm position, stamina, alternating-input state, AI cycle timers, warnings, temporary bonuses, and all other round-specific state.

No game mechanics continue updating after Game Over.

Existing arm rendering, clasped hands, table arena, and post-game screen remain functional.

Step 7: Test and Review

After implementing the changes:

Run any existing tests and relevant linting or syntax checks.

Add or update tests if the project has a test framework.

If automated gameplay tests are unavailable, validate the movement and state-transition logic through focused checks.

Test the sign and direction of player movement explicitly.

Test exhaustion, stamina recovery, AI surge transitions, counter-surge activation, bonus expiration, victory/loss boundaries, and rematch reset.

Check for undefined variables, incorrect state transitions, duplicated input handling, and timer bugs.

Review the final changes to ensure that all four tasks are completed together and do not conflict.

Do not claim a test passed unless you actually ran it. If you cannot launch the game or run a particular test, clearly state that limitation.

Implementation Guidelines

Follow the project's existing programming language, libraries, coding conventions, and architecture.

Avoid unnecessary dependencies and unrelated refactoring.

Use clear, configurable constants for stamina thresholds, AI force multipliers, surge/cooldown durations, counter-surge timing, bonus duration, and recovery rates.

Keep the game loop non-blocking.

Use elapsed-time-based updates where appropriate rather than frame-dependent timers.

Avoid magic numbers scattered throughout the code.

Handle keyboard events and game-over states safely.

Ensure all new state is initialized and reset correctly.

Keep changes limited to the necessary files.

Final Deliverables

After completing the work, provide:

A concise explanation of the original inverted movement bug and its fix.

A summary of the changes made for Tasks 1–4.

The list of files modified.

The tests and checks actually performed, including their results.

Any remaining issues or manual testing steps.

Important: Implement and verify the actual code changes in the repository. Do not stop after presenting a plan, explaining what should be changed, or providing isolated code snippets. Complete all four tasks

while preserving the existing working game. Dont make changes directly just return the updated code file

Read 3 files, ran 4 commands

pygame can't be installed here because the network blocks PyPI. I'll write the engine now and test its logic against a small stub of pygame.

Created a file, shared a file, ran 4 commands

I've written the updated game/game_engine.py and attached it; main.py needs no changes. I didn't touch your repo. I couldn't launch the real game because pygame wouldn't install here (the network blocks PyPI), so I only tested the game logic.

1. The bug. Both the Left and Right key handlers did arm_position += 4.2. That pushed the hands toward +100, the computer's side, so the player pinned themselves. Both now do arm_position -= strength. Alternation is still required, and a repeated key, an exhausted press or any other key does nothing. The position is clamped to ±100 and the win check runs right after each push.
2. AI surge. The AI now cycles through normal (4s), surge (2s) and cooldown (3s) using elapsed time, so it never blocks the loop. It keeps its existing force and random variance. A surge multiplies that force by 1.8 and a cooldown by 0.35. All of these are constants at the top of the file.
3. Warnings.

AI banner: a banner under the stamina bar shows the AI phase. It's a blinking red "AI SURGE" with a countdown, a blue "AI COOLDOWN", or grey "normal".

Exhaustion: below 10 stamina an orange "EXHAUSTED" banner appears and pushing is blocked. You can push again once stamina recovers to 25, so mashing keys can't bypass it.

Existing UI: the arena and stamina bar are unchanged.

4. Counter-surge.

Trigger: the window opens for 0.6s at the moment the surge ends. A valid push by a non-exhausted player inside it triggers the bonus once.

Effect: for 4s, push strength is ×2 and stamina recovery is ×3. A gold banner and a gold stamina bar show it's active, and a green "PUSH NOW!" prompt shows during the window.

Reset: the bonus can't re-trigger while active or after the window closes, and it is cleared on game over. Pressing R resets everything.

Changes you didn't ask for:

Stamina recovery is now 15 per second instead of 48. At 48 a player basically could never get exhausted, so the exhaustion feature would never show.

The engine now tracks time itself, so update() takes an optional dt and main.py stays as it is.

Tests I ran.

Setup: a script of about 40 assertions against a small pygame stub. It ran with py_compile, and all assertions passed.

Movement: direction and sign, alternation and ignored repeats, and ±100 bounds.

AI: the phase order and force ordering (cooldown < normal < surge).

Player state: exhaustion block and recovery, and that spamming keys can't bypass it.

Counter-surge: double strength, ×3 recovery, expiry, no re-trigger, and no trigger when exhausted, invalid, outside the window, or during surge or normal.

Game over and rematch: victory and loss, no updates after game over, and that R resets every field.

Rendering: the rendering code runs in every state, but against a stub, so it was never actually drawn.

Still to do on your side.

Run pip install pygame and python main.py. Check that the banners look right and don't overlap, and that the game feels fair.

Idle, the computer wins in about 5 seconds. If it feels too hard, lower AI_SURGE_MULTIPLIER or AI_BASE_FORCE_PER_SEC.

The README has no tests folder, so I didn't add any to the repo. The test script is in my scratchpad if you want it.

Game engine

PY

Claude is AI and can make mistakes.

Claude finished the response